

Parallel Programing

By Josef Arango, Sage Cauley, Wesley Simms

Abstract

This paper will talk about how we can use parallel computing to complete a task. The parallel and distributed applications perform on a real system on Beocat is the High-Performance Computing (HPC) cluster at Kansas State University. We will be developing a parallelized program using the three standard tools for building this program: pthreads, MPI, and OpenMP. We might be using CUDA maybe if we want to feel bad

Introduction:

Implementation:

Pthreads-

For our Pthread implementation we took in 2 arguments to our main function for running the program that is first the file to run and second how many cores to use for that specific run. From there it reads the files line by line and stores them dynamically into memory. After that it initializes threads and assigns them a portion of lines from the file to process. It will then wait for all the threads to get finished and then print out either the time it takes or the max character of each line along with a line number. To finish it all we clean up all the memory and anything extra.

MPI-

OpenMP-

Evaluation:

Pthreads-

When evaluating each of the threads we did the following tests found in table 1. Each of the tests were run a total of 5 times to get an average of the processing time. Each of these tests were done with a total memory of 7GB to have more than enough space for the files to be loaded into memory. All the tests were restricted to having only one thread per core and constrained to the Mole compute node.

1 Core	5 Cores	10 Cores	15 Cores	20 Cores
10 Cores Between 2 systems	20 Cores Between 2 Systems			
5 Cores With 1K Lines	5 Cores With 10K Lines	5 Cores With 100K Lines	5 Cores With 1M Lines	

Below you will find the first row of tests. When more cores were added the times drastically decreased the system time. As we got closer to 20 cores the time started to plateau. This is due to the cost of creating a new thread that could have been used to actually run the

program and taking valuable compute time from the system.

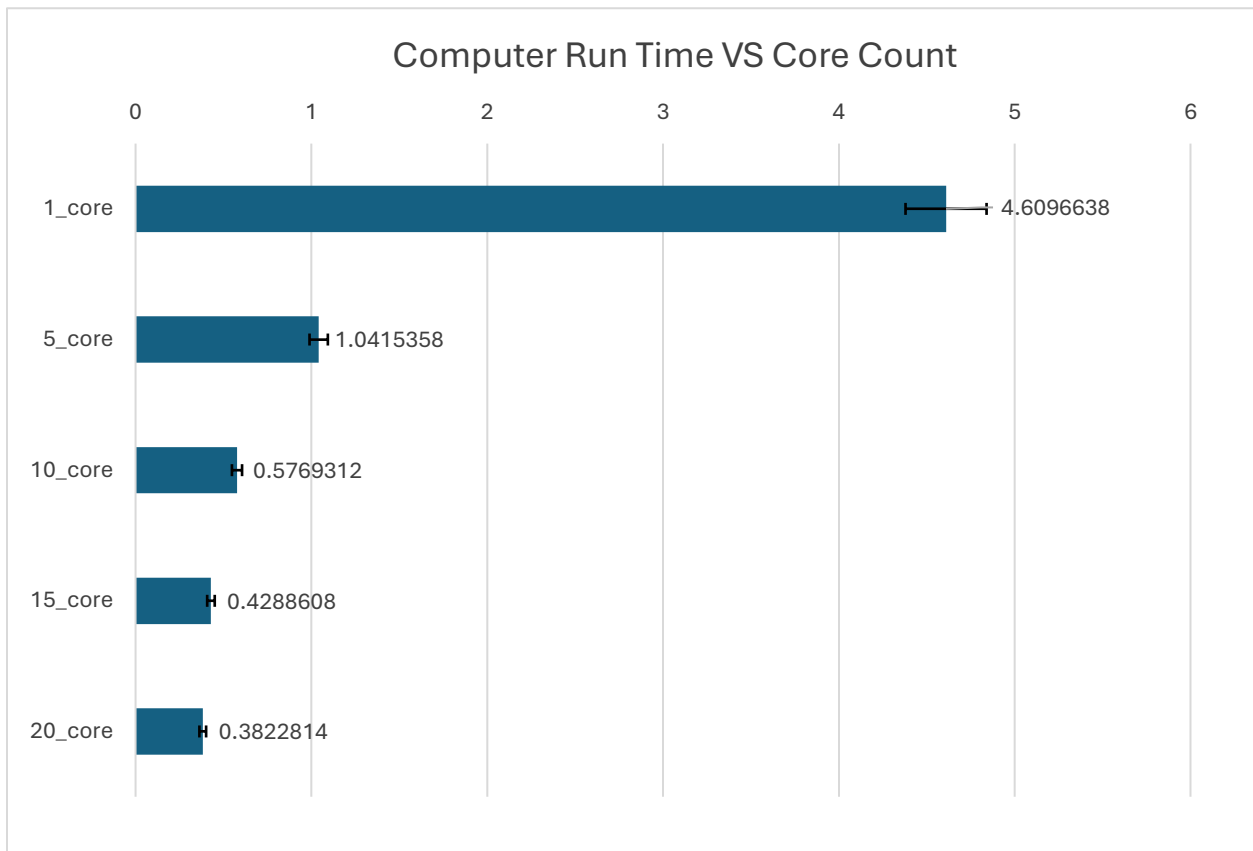


Figure 1: Computer Run Time VS The Core Count Used

The following graph shows the difference between 1 system and 2 system. All other variables were kept the same to limit the differences and get clean differences. Looking at the data it is noted that the 2 systems took longer than running on one system. This is to be expected as the process must transfer data between the two systems that will never be faster than on chip interconnect speeds. Note that 10 is somewhat close as it was somewhat reasonable to use two systems as the scheduler time was not significant enough to make a difference. Then with 20 cores it took over 2x the time caused by the information transfer.

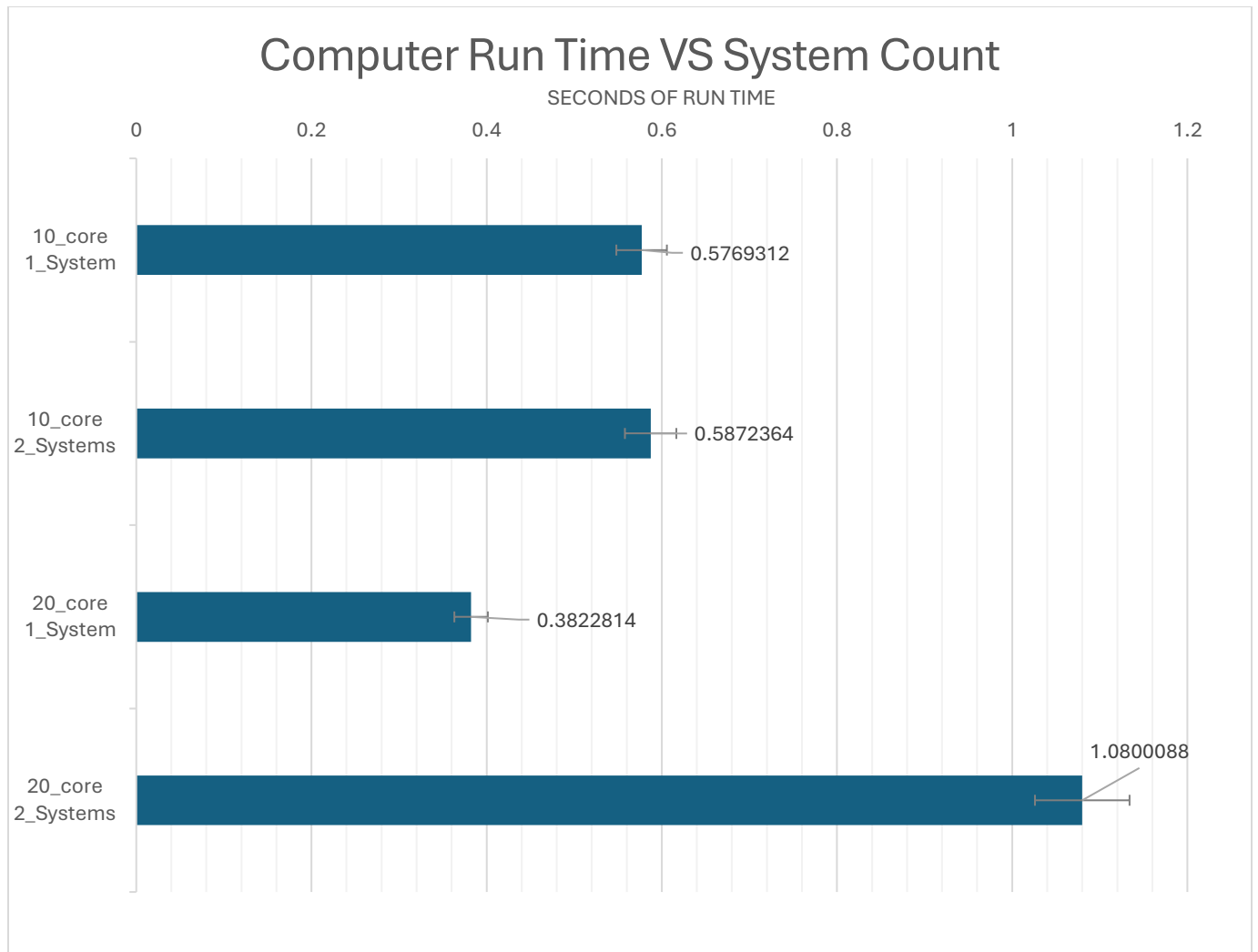


Figure 2: The Compute Run Time Between 1 and 2 systems

The following figure is the different file sizes. All these tests were done using 5 cores and the default configuration. Each of the files was built using a python script found in the appendix. The different sizes had the same number of elements in each line but the number of lines were changed to 1k, 10k 100k and 1M lines. Then each of the 5 tests were run on each of the files. Looking at the graph it is evident that the number of the element's changes in the INSTERT TERM HERE in time.

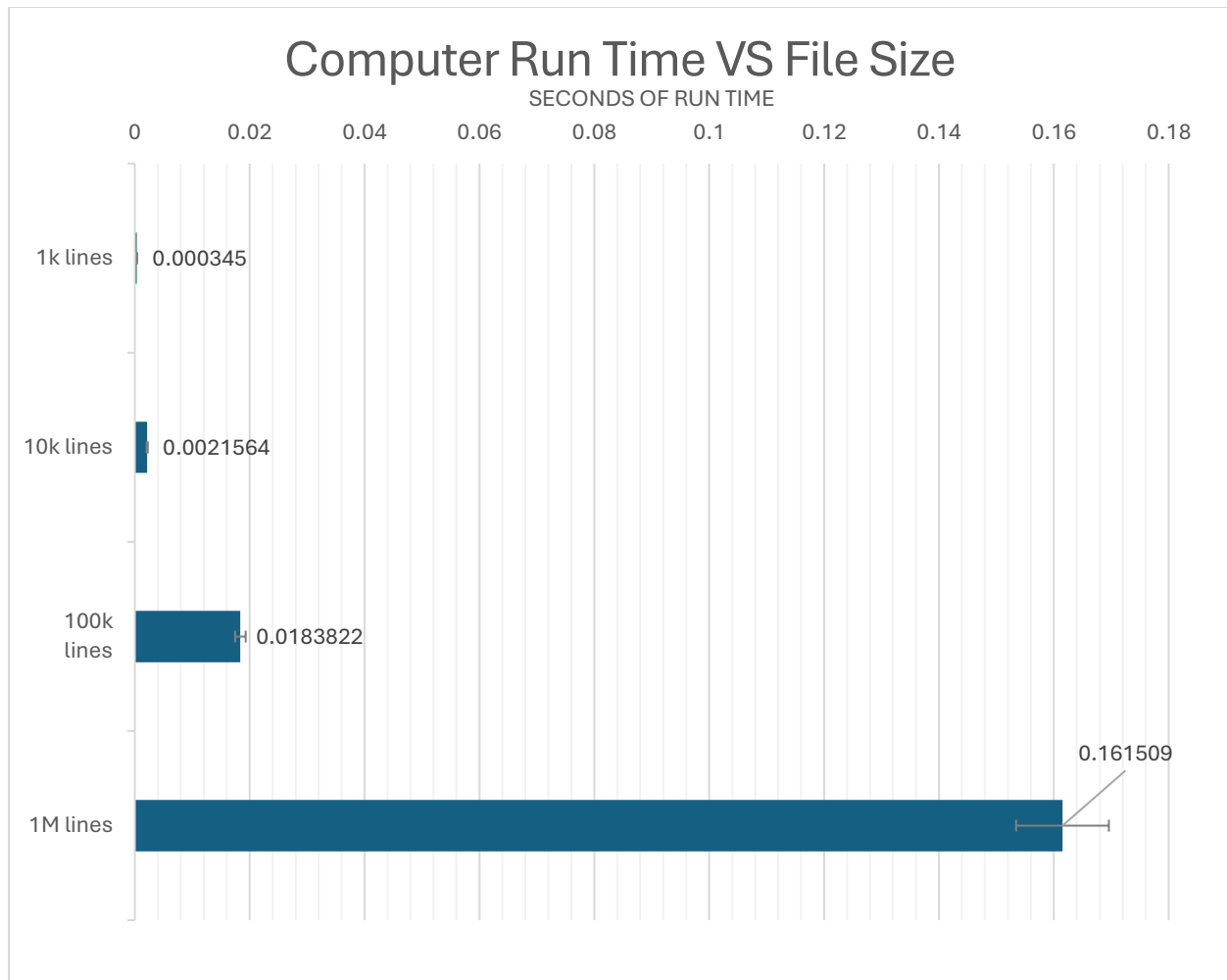


Figure 3: The Computer Run Time VS The Size of the File

MPI-

OpenMP-

Conclusion:

STUFF WORKS

References:

Appendix:

Using the stuff, we use the devil to make the code. We had to give up Sage's first born, Josef's prized goat and Wesley's left kidney. Sage took most of the fall as he said he got this. After the summon ritual was complete, we had the code to test.